

Desenvolvimento guiado por especificação

IA nos jogos, e como usá-la sem perder o controle

ROTEIRO

Quatro partes, nesta ordem

1

O contexto

IA na indústria e
MCPs nos motores

2

O que temos hoje

Vibe coding,
harness e
especificação

3

A ideia do fluxo

O custo, e quando
vale a pena

4

Na prática

O Zé Peixeira, do
requisito ao teste

Quem aqui já pediu código de jogo a uma IA?

E o que aconteceu depois?

A IA NOS JOGOS HOJE

Quanto a indústria de jogos usa IA

36%

usam IA generativa no trabalho

GDC 2026 · mais de 2.300 profissionais

47% dos que usam: para código

90%

usam alguma forma de IA no trabalho

Google Cloud e Harris Poll, 2025 · 615 desenvolvedores de 5 países

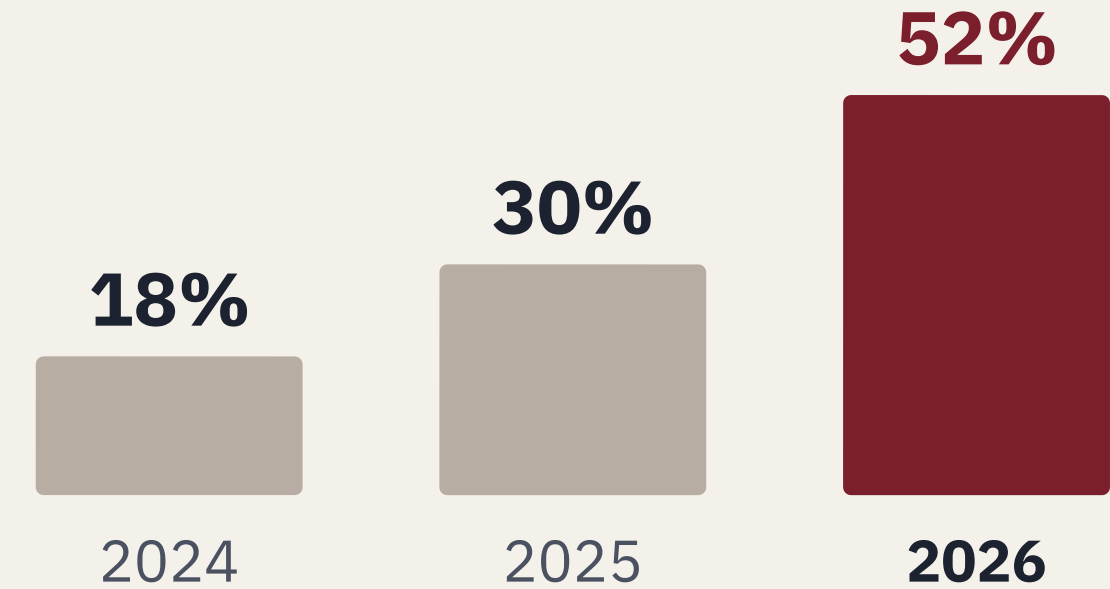
**Por que 36% numa
pesquisa e 90% na outra?**

As duas falam da mesma indústria, no mesmo ano.

A IA NOS JOGOS HOJE

O uso cresce, e a rejeição também

Acham que a IA generativa prejudica a indústria



Por área, em 2026

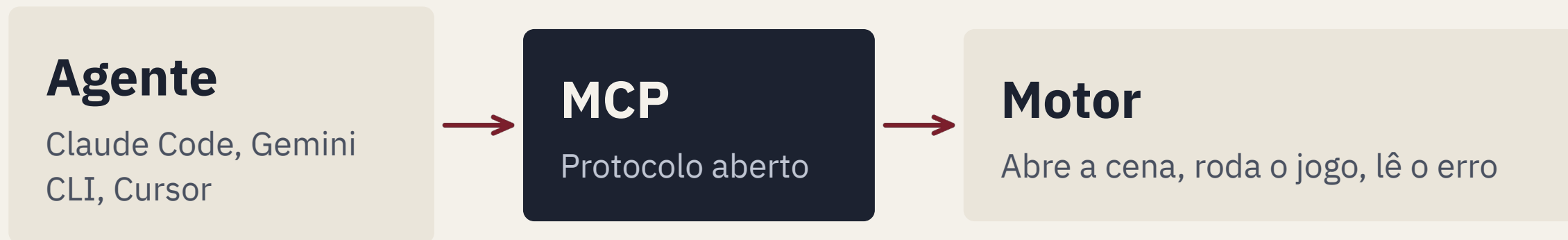
Arte	64%
Design e narrativa	63%
Programação	59%

Visão positiva: 7% (13% em 2025)

Fonte: GDC State of the Game Industry 2024, 2025 e 2026

OS MOTORES ENTRAM NA ERA DOS AGENTES

MCP: o motor vira ferramenta



Com MCP, o agente roda o jogo, lê o erro e corrige sozinho: mais autonomia, e mais revisão.

OS MOTORES ENTRAM NA ERA DOS AGENTES

MCP nos motores, hoje

Motor	Quem faz	Situação
Unity	Unity	Oficial · beta · pago
Unreal 5.8	Epic	Oficial · experimental · grátis
Godot	Comunidade	Vários projetos, nenhum oficial
Blender	Comunidade	blender-mcp, não é do Blender

Os comerciais oficializam. O livre depende da comunidade.

O QUE TEMOS HOJE

Quatro jeitos de programar com IA

Autocompletar

Sugere a próxima linha

Copilot

Conversa

Você pergunta, copia e cola

ChatGPT, Gemini

Agente

Edita arquivos e roda comandos

Claude Code, Gemini CLI

Gerador de app

Descreve e recebe pronto

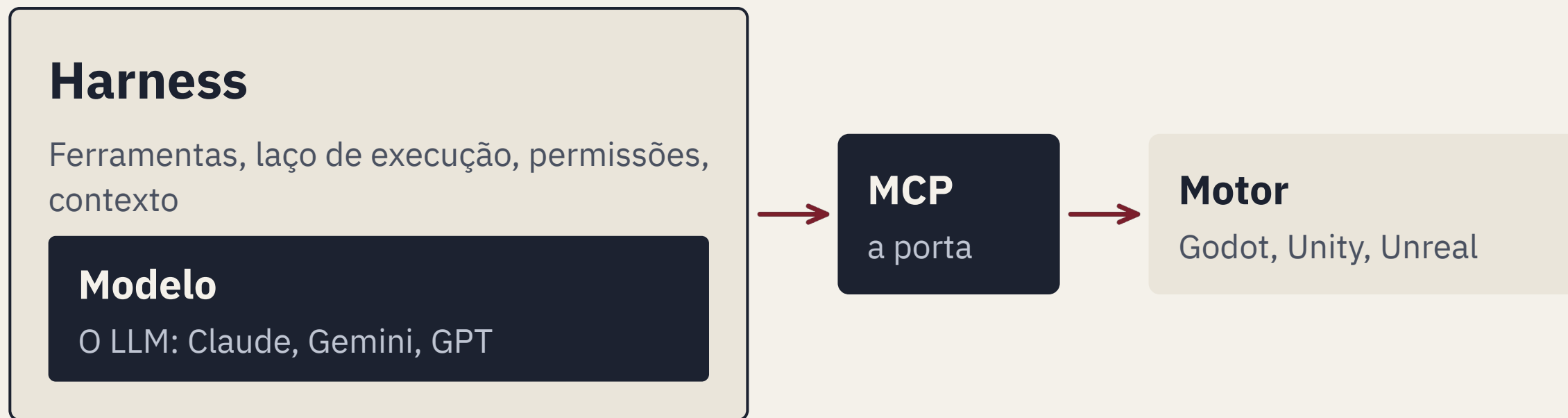
Lovable

Mais controle seu

Mais autonomia da IA

O QUE TEMOS HOJE

Modelo, harness e MCP



Claude Code e Gemini CLI são harnesses: decidem o que o modelo pode ler, rodar e alterar.

O QUE TEMOS HOJE

Vibe coding

“forget that the code even exists”

Andrej Karpathy, fevereiro de 2025

Descrever, aceitar o que vem, rodar e repetir. Ótimo para protótipos, até ninguém conseguir explicar o código.

Você pode entregar código que a IA escreveu; não pode entregar código que não consegue explicar.

O QUE TEMOS HOJE

Onde o vibe coding quebra

Perde o contexto

Em conversas longas, esquece o que já foi decidido.

Inventa API

Usa KinematicBody2D, do Godot 3, onde o Godot 4 pede CharacterBody2D.

Mistura escopo

Pedi só a corrida; vieram corrida, pulo e dash, sem teste.

Código sem dono

Funciona, mas ninguém sabe dizer por quê.

O QUE TEMOS HOJE

Por que especificar ganhou força

Escrever o que o jogo deve fazer antes de pedir o código.

Ganhou tração em 2025 e 2026. Não há dado de que seja a forma mais usada.

Spec Kit · GitHub

Aberto; funciona com vários agentes.

Kiro · AWS

Editor com a especificação embutida.

Então por que ainda fazer vibe coding?

O que ele tem que a especificação não tem?

A IDEIA DO FLUXO

1

Especificação

Critérios em que dá para dizer "passou" ou "não passou"

2

Passo pequeno

Cada tarefa termina com o jogo rodando

3

Verificação

Testes e leitura humana antes de aceitar

Especificar não impede o agente de alucinar: reduz a margem. Quem pega o erro é a verificação.

A IDEIA DO FLUXO

Por que Godot

Gratuito e aberto

Sem licença e sem conta; roda no laboratório.

Tudo em texto

Cenas e scripts que o agente lê e o Git compara.

GDScript tipado

Os tipos ajudam o agente e a revisão.

Testes sem janela

O GUT roda os testes pela linha de comando.

E o Unity? A spec não muda; mudam a constituição, o plano e as tarefas.

A IDEIA DO FLUXO

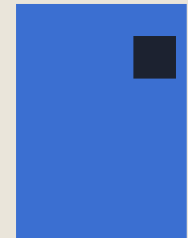
Estudo de caso: Zé Peixeira

O que é

- Controle de personagem 2D inspirado em Celeste
- No sertão cearense
- Seis fatias, da corrida ao corte

O que não é

- Um jogo completo
- Game design ou arte
- Roteiro obrigatório do trabalho



O Zé, 8×11 px

A peixeira é ferramenta, não arma. O gibão de couro protege dos espinhos.

É CARO, SIM

O custo da fatia 001

11

tarefas, cada uma
aprovada

6

problemas
registrados

17

testes com o
requisito no nome

1/6

fatias feitas

O que compra: controle, qualidade e rastreabilidade.

Fluxo completo ou vibe coding?

1. Protótipo de game jam, em 48 horas
2. Página de divulgação
3. Save de um jogo com atualizações
4. Controle do personagem principal
5. O trabalho final da disciplina

Para cada caso: qual, e por quê?

NA PRÁTICA

Os artefatos do fluxo

Artefato	Quem escreve	Depende do motor?
Constituição	Humano	Sim
Spec	Humano	Não
Plano	Agente, revisado	Sim
Tarefas	Agente, revisado	Sim
Código e teste	Agente, revisado	Sim

Se a spec cita `CharacterBody2D`, ela vazou implementação.

NA PRÁTICA

Os comandos do Spec Kit

/speckit.constitution	revisa as regras do projeto
/speckit.specify	cria a spec (aqui, escrita pelo humano)
/speckit.clarify	até 5 perguntas sobre ambiguidades
/speckit.plan	plano técnico a partir da spec
/speckit.tasks	divide o plano em tarefas pequenas
/speckit.analyze	confere spec, plano e tarefas entre si
/speckit.implement	implementa (aqui, uma tarefa por vez)
/speckit.converge	procura lacunas depois de implementar

NA PRÁTICA

Uma tarefa, um projeto

Pasta	O que observar ao apertar F5
T001	A fase e o Zé parado; nenhuma regra ainda
T003	As setas movem o Zé; acelera e freia
T005	Sai andando da plataforma e cai
T007	O olho troca de lado antes da velocidade
T008	Cair no vão traz o Zé de volta
T011	Os testes instanciam a fase inteira

NA PRÁTICA

O que a revisão pegou

Clarify parou cedo

1 pergunta, de até 5. Na nova rodada, mais 4 decisões.

Teste vazio, T003

Passava 0 direto para a função. Não tinha como falhar.

Teste vazio, T011

Passava mesmo sem a fase chamar o reaparecimento.

Seis registros em docs/review/: pedido, resultado, como o erro foi percebido, o que mudou.

**A especificação reduz o
erro. A verificação o
detecta.**

PARA O TRABALHO FINAL

O que do seu jogo merece especificação?

Material

Repositório ze-peixeira, no
GitHub SMD-Jogos

Bônus

Refazer com o Gemini CLI